

LLMsVerifier

LLMsVerifier

검증. 모니터링. 최적화.

요약

LLMsVerifier는 여러 제공업체에 걸친 대규모 언어 모델(LLM)을 검증, 모니터링, 최적화하기 위한 엔터프라이즈급 플랫폼으로, “내 코드를 보시나요?”라는 필수 검증 테스트를 기반으로 구축되었습니다. 이 테스트를 통과한 모델만 사용 가능 또는 내보내기 대상으로 표시됩니다.

간략 설명

여러 제공업체의 LLM을 검증, 벤치마킹, 모니터링, 최적화하는 Go 플랫폼입니다. 모든 모델은 사용 전에 필수 코드 가시성 테스트를 통과해야 하며, 이후 지연 시간, 스트리밍, 함수 호출, 비전, 임베딩 검사를 수행하고, 검증된 구성만 AI 및 CLI 도구로 내보냅니다.

상세 설명

LLMsVerifier는 여러 제공업체에 걸친 LLM 성능을 검증, 모니터링, 최적화하기 위한 종합 플랫폼입니다. 핵심 원칙은 필수 검증이며, 이에 대해서는 타협하지 않습니다. 어떤 모델이든 사용 가능으로 표시되거나 내보내기 구성에 포함되기 전에 반드시 “내 코드를 보시나요?” 테스트를 통과해야 합니다. 이 테스트는 제공업체에 실제 HTTP 요청을 보내 응답을 분석해, 그럴듯한 에코가 아닌 진정한 이해력을 확인합니다. 입력 내용을 실제로 보고 이해할 수 없는 모델은 결코 “사용 가능” 플래그를 얻을 수 없습니다. 이 관문을 통과한 후 검증 엔진은 존재 여부, 응답성, 지연 시간, 스트리밍, 함수 호출, 비전, embeddings 등 다양한 기능 테스트를 수행하며, 리포터 엔진은 그 결과를 마크다운 및 JSON 보고서로 변환해 즉각적인 조치가 가능하도록 합니다.

이 시스템은 모듈형이며 이벤트 기반으로 설계되어, 검증 엔진, 리포터 엔진, 구성 관리자를 핵심으로 CLI, TUI, 웹, REST 및 API 인터페이스를 제공합니다. 검증에 그치지 않고, 고급 레이어에서는 LLM 기반 작업 분해를 위한 감독자/작업자 패턴, 매우 긴 세션에서도 컨텍스트 유실을 방지하는 슬

라이딩 윈도우 및 LLM 요약 기반 컨텍스트 관리, 클라우드 기반 체크포인팅, 서킷 브레이커와 지연 시간 기반 라우팅을 갖춘 장애 조치 시스템을 제공합니다. 주변 인프라는 프로덕션 환경에 최적화되어 pub/sub 이벤트 버스, 크론 스케줄링, 가격/제한 감지, RAG용 vector 데이터베이스, 내보내기 시스템으로 구성됩니다. 또한 생성된 모든 제공업체/모델 이름에는 (llmsvd) 접미사가 자동으로 추가되어, 검증된 출력이 한눈에 식별 가능하며 검증되지 않은 모델과 혼동될 염려가 없습니다. 검증된 모델만 AI 및 CLI 도구(OpenCode, Crush, Claude Code 등)의 내보내기 구성에 기록됩니다. 실제 프로덕션 환경에서 필요한 운영 도구도 함께 제공됩니다. Docker/Kubernetes/Helm 배포, Prometheus/Grafana 모니터링, LDAP/SSO, SQLCIPHER 암호화 스토리지 등이 포함됩니다.

개발 배경

구성 파일만으로는 신뢰할 수 없기 때문입니다. API 키가 만료되거나 모델이 더 이상 지원되지 않을 수 있으며, 구성 파일은 실제 지연 시간, 실제 오류, 모델이 실제로 입력 내용을 보고 이해할 수 있는지에 대해서는 아무런 정보를 제공하지 않습니다. LLMsVerifier는 “구성에 있으니까 작동할 것이다”라는 가정을 배제하고, 실제로 올바르게 응답하는 모델만 사용 가능으로 표시하고 내보냅니다.

콘텐츠

왜 혁신적인가

LLM 플릿에 신뢰성을 부여합니다. 이는 설정만으로는 결코 얻을 수 없는 단어입니다. 수많은 설정들이 의도적으로 정보를 누락하며 거짓말을 일삼는 공간에서, 이 시스템은 구성된 모델이 제대로 작동하기를 바라는 수준을 넘어, 모든 모델이 실제 검증을 통과했다는 강제 가능하고 테스트 가능한 보장을 제공합니다. 모니터링, 장애 조치, 검증된 모델만 내보내는 기능을 통해 검증부터 운영까지의 전체 사이클을 완성합니다. Helix 생태계 내에서 이 시스템은 LLM 모델, 제공자, 검증 메타데이터에 대한 유일한 신뢰할 수 있는 정보원이 됩니다. 다른 서비스들(HelixTranslate 포함)은 이 시스템을 기준으로 라우팅되므로, “지금 실제로 작동하는 모델은 무엇인가?”라는 질문에 하나의 정확한 답변을 제공합니다. 각 팀이 각자의 희망 섞인 추측을 유지하는 대신, 플랫폼 전체가 하나의 진실을 공유하게 됩니다.

혁신적인 점

- 강제 “내 코드를 보시겠습니까?” 검증 — 모델이 사용 가능해지기 전에 반드시 통과해야 하는 실제 HTTP 기반의 이해력 검증 게이트로, 제품의 핵심 차별점이자 검증되지 않은 모델이 절대 유입되지 않는 이유입니다.
- 검증된 모델만 내보내는 설정 — AI CLI 도구용으로 생성된 설정에는 검증에 통과한 모델만 포함되므로, 배포된 설정에서 조용히 문제가 있는 모델이 다시 유입될 위험이 없습니다.

- **(llmsvd)** 브랜딩 접미사 시스템 — 생성된 모든 제공자/모델에는 추적 가능한 접미사가 붙으며, 이로 인해 검증된 출처가 출력물이 이동하는 모든 곳에서 명확히 드러납니다.
- 다양한 **CLI** 에이전트 및 제공자 간 기능 감지 — 스트리밍 유형(SSE, WebSocket, JSONL, EventStream), 압축, 캐싱 동작 등을 가정하지 않고 실제 감지합니다.
- 탄력적인 장애 조치 — 회로 차단기, 응답 지연 기반 라우팅(첫 토큰 생성 시간이 임계치를 넘으면 재라우팅), 상태 확인, 가중치 기반 트래픽 분배 등을 통해 개별 제공자가 불안정해져도 플릿 전체의 응답성을 유지합니다.
- 장시간 자율 운영 — Supervisor/Worker 분해 패턴과 체크포인팅, 메모리 통합을 통해 컨텍스트가 소진될 위험이 있는 장기 세션도 안정적으로 유지합니다.
- **RAG / vector-DB** 통합을 통한 맥락 강화 지원.

가장 큰 기술적 도전 과제와 해결 방법

- 모델이 단순히 설정되었을 뿐인지, 실제로 작동하는지를 증명하는 것. 이것이 핵심이자 가장 어려운 부분이었습니다. 실제 API 호출을 통해 모델의 응답을 분석하고 광범위한 기능 테스트를 거친 뒤, 검증에 실패한 모델은 내보내지 않는 방식으로 해결했습니다. 즉, 설정 자체가 아닌 검증이 운영의 관문이 됩니다.
- 불안정한 수많은 서드파티 제공자와의 신뢰성 확보. 제공자의 불안정성을 정상적인 상황으로 간주하는 장애 조치 오케스트레이터로 해결했습니다. 회로 차단기는 N회 실패 시 M초 이내에 제공자를 저하 상태로 전환하고, 지연 기반 라우팅은 느린 엔드포인트를 회피하며, 주기적인 상태 확인으로 복구 여부를 탐지하고, 가중치 라우팅으로 비용 효율적인 모델과 프리미엄 모델을 균형 있게 분배합니다.
- 매우 긴 자율 세션 유지. 대규모 작업을 처리 가능한 단위로 분할하는 Supervisor/Worker 분해 패턴과, 중단 시에도 진행 상황을 유지하는 클라우드 스토리지 체크포인팅, 계층화된 컨텍스트 관리(슬라이딩 윈도우 + LLM 요약 + RAG)를 통해 해결했습니다. 이를 통해 모델은 토큰에 압도되지 않으면서도 맥락을 유지할 수 있습니다.
- 제공자 확산 문제. 하나의 공통 인터페이스 뒤에 수많은 제공자별 Go 어댑터를 숨기고, 실제 엔드포인트를 중앙에서 관리하는 방식으로 해결했습니다. 따라서 새로운 제공자를 추가하는 것은 코드 전체에 영향을 미치는 변화가 아닌, 제한된 변경으로 가능합니다.

콘텐츠

기술 스택

- **Go** — 동시성 처리에 강점을 가진 핵심 플랫폼 언어로 선택되었으며, 다중 스레드 검증 엔진을 구동해 여러 모델을 병렬로 검사할 수 있으며 주변 서비스도 지원합니다.
- **Gin** — REST API 서버로 선택되었으며, JWT 인증, 요청 제한, WebSocket/SSE 엔드포인트를 담당합니다.

- **SQLite + SQLCipher** — 검증 데이터(키, 결과 등)가 민감하여 기본적으로 저장 시 암호화가 필요한 만큼, 데이터베이스 수준의 암호화를 지원하는 임베디드 스토리지로 선택되었습니다.
- **Redis** — 빈번한 검증 및 메타데이터 조회 속도를 높이기 위한 캐싱 계층으로 선택되었습니다.
- **RabbitMQ + Kafka** — 플랫폼 전반에 걸쳐 생산자와 소비자를 분리하는 이벤트 기반 아키텍처를 구현하기 위한 메시징 및 스트리밍 기술로 선택되었습니다.
- **gRPC + Protocol Buffers** — 구성 요소 간 강력한 타입의 서비스 간 통신 및 이벤트 전송을 위해 선택되었습니다.
- **QUIC / HTTP-3 (quic-go)** — 최신 전송 프로토콜 지원(문서상 HTTP/3 제공업체 가용성은 제한적이며, 이는 제공 가능한 기능이지 보편적인 주장은 아님)을 위해 선택되었습니다.
- **JWT + LDAP/NTLM** — 기업 인증을 지원해 기존 기업 ID 체계(SSO/SAML/OIDC 등)에 자연스럽게 통합될 수 있도록 선택되었습니다(문서상 지원 여부 명시됨).
- **Viper(설정), Logrus(로깅), Brotli/compress(압축)** — 유연한 설정, 구조화된 로그, 페이로드 압축 등 운영 인프라를 담당합니다.
- **Angular** — 검증 및 모니터링의 시각적 진입점인 웹 싱글 페이지 애플리케이션을 구현하기 위해 선택되었습니다.
- **Python + JavaScript SDK** — 클라이언트 팀에게 일급 접근성을 제공하며, OpenAPI/Swagger를 통해 문서화됩니다.
- **Docker, Kubernetes, Helm** — 헬스 모니터링 및 자동 스케일링을 지원하는 프로덕션 배포를 위해 선택되었으며, 검증 플릿이 현대적인 서비스로 확장될 수 있도록 합니다.
- **Prometheus + Grafana** — 메트릭 및 대시보드를 제공해 플랫폼 자체의 상태를 모니터링하는 모델만큼 투명하게 관리할 수 있도록 선택되었습니다.
- **Testify(Go) + node -test/jsdom(웹)** — Go 코어와 웹 프론트엔드를 아우르는 계층적 테스트를 위해 선택되었습니다.

현황 및 솔직한 메모

- **현황:** 베타. Go 소스는 실제 HTTP 검증을 구현하고 있습니다(검증을 설정 전용으로 설명하는 레거시 문서는 목표에 불과하며 구식이므로, 코드가 공식 기준입니다).
- **라이선스:** 미정. README에는 MIT로 명시되어 있으나 Dockerfile 레이블에는 Apache-2.0으로 표기되어 있어 배포 전에 확정해야 합니다.
- **제공업체 수:** README에는 “12개 어댑터”라고 명시되어 있으나 providers 디렉터리에는 약 26개가 나열되어 있습니다. “12개 이상 / 추가 개발 중”으로 간주하시기 바랍니다. 다수의 “FINAL/COMPLETE” 상태 파일은 존재하지만, 코드, 문서, go.mod가 공식 기준입니다.
- **저장소:** vasic-digital 조직에 속해 있으나, 기능적으로는 Helix LLM 인프라 클러스터의 신뢰 계층 역할을 합니다.

우선순위 계층: Helix-주요(LLM 인프라 클러스터; LLM/제공업체/검증 메타데이터의 단일 정보원). HelixTrack 다음으로 우선합니다.